

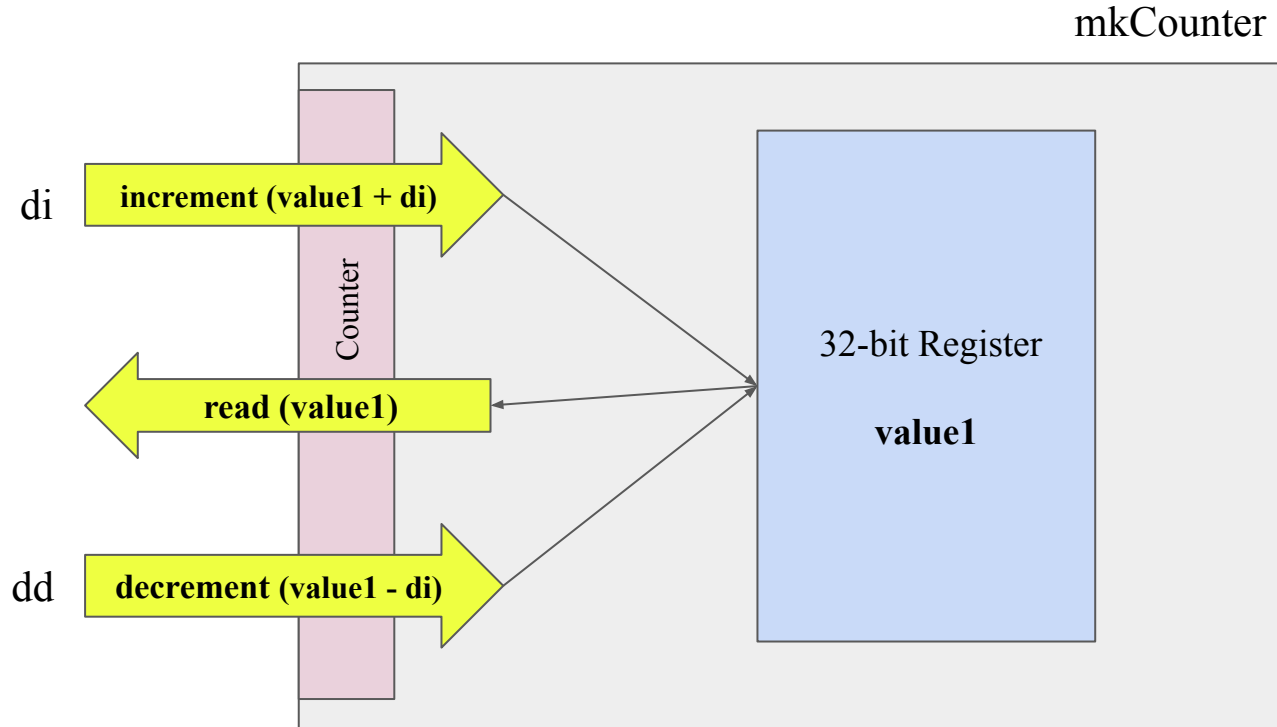
CS6230

BLUESPEC DESIGN DEMO

Up/Down Counter and RSA

UP/DOWN COUNTER

Counter design



How to handle conflicts while accessing methods?

- Counter operates on int values (32 bits)
- Initial value of Register : value1 = 0
- Pass the increment/decrement values (di/dd) using increment/decrement methods
- Read the register value using read method
- increment, decrement methods : write onto value1 register
- read method : reads from value1 register
- So, cannot call both increment and decrement methods simultaneously in a single rule - will cause conflict
- Then, how to instantiate and use both methods of mkCounter?

Handling method conflicts

```
// Instantiate the counter
Counter myCounter <- mkCounter_v1;

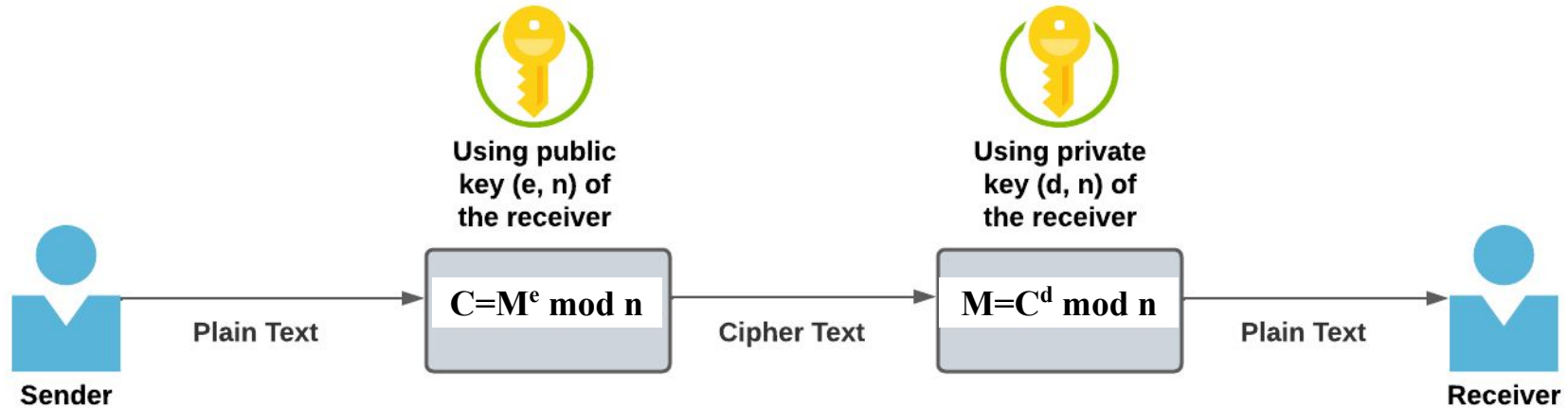
// Register to track the current mode: 0 for counting up, 1 for counting down
Reg#(Bool) countingUp <- mkReg(True);

// Rule to increment the counter by 1 each cycle if counting up
rule up_counter (countingUp && myCounter.read() < 10);
  myCounter.increment(1);
  let value = myCounter.read();
  $display("Counter Value (Up): %0d", value);
  if (value == 9) begin
    countingUp <= False; // Switch to counting down on next cycle
  end
endrule

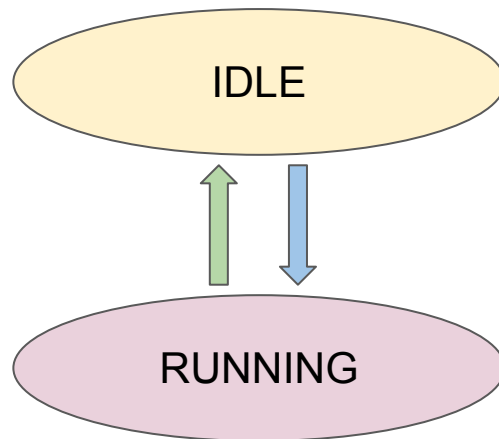
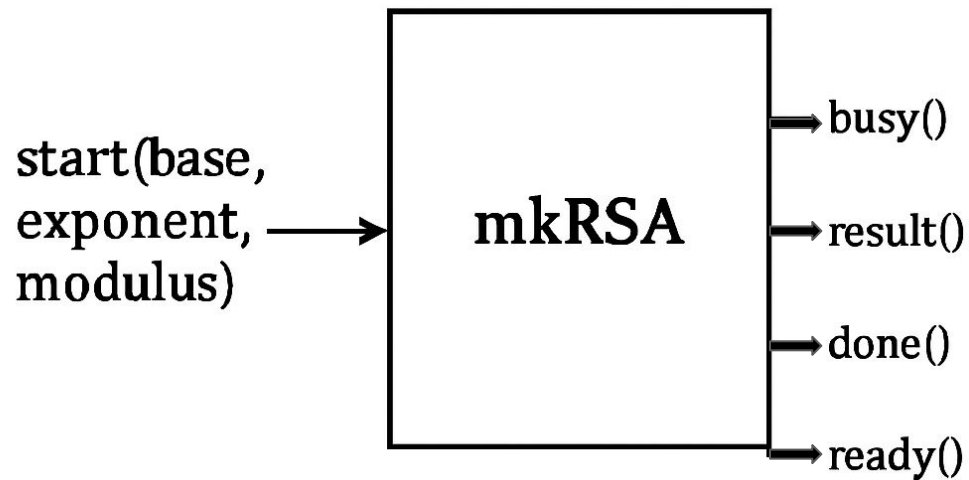
// Rule to decrement the counter by 1 each cycle if counting down
rule down_counter (!countingUp && myCounter.read() > 0);
  myCounter.decrement(1);
  let value = myCounter.read();
  $display("Counter Value (Down): %0d", value);
  if (value == 1) begin
    countingUp <= True; // Optionally reset or end simulation
    // You could end the simulation here with '$finish' or reset to start over
    $finish;
  end
endrule
```

RSA

RSA : A public-key cryptography algorithm



Design



mkRSA - Code Flow

$$\mathbf{result = base^{exponent} \bmod modulus}$$

- The module effectively computes the result by iterating over each bit of the exponent, making it efficient in hardware while avoiding large multiplications.
- For each bit of the exponent:
 - Multiply result when bit = 1
 - Always square the base
 - Stop after all bits are shifted out

Walkthrough:

Given : **base = 7** **exponent = 13 (binary 1101)** **modulus = 33**

1. Initialize $\text{res} = 1$, $\text{base} = 7 \% 33 = 7$

2. Bit 0 = '1' $\rightarrow \text{res} = (1 * 7) \% 33 = 7$

Square base $\rightarrow 7^2 \% 33 = 16$

Shift exp = 110

3. Bit 1 = '0' $\rightarrow$ no multiply

Square base = $16^2 \% 33 = 25$

Shift exp = 11

4. Bit 2 = '1' $\rightarrow \text{res} = (7 * 25) \% 33 = 10$

Square base = $25^2 \% 33 = 31$

Shift exp = 1

5. Bit 3 = '1' $\rightarrow \text{res} = (10 * 31) \% 33 = 13$

Square base = $31^2 \% 33 = 4$

Exponent now 0 $\rightarrow$ stop

Final result of $7^{13} \bmod 33 = 13$